

Chapter 9

Machine-independent Optimizations

Code Optimization

Code Optimization

- High-level language constructs can introduce substantial run-time overhead if naively each construct is independently translated into machine code
- Code optimization or code improvement
 - Elimination of unnecessary instructions in object code
 - Replacement of one sequence of instructions by a faster sequence of instructions

Code Optimization

Code Optimization

- Local Optimization

- Code improvement within a basic block
- Covered in chapter 8

- Global Optimization

- Code improvement across basic blocks
- Focus of this chapter

Global Optimizations

Global Optimizations

- Most global optimizations are based on **data-flow analysis (DFA)**
- DFA are algorithms that gather information about a program
- For each instruction in program, specifies some property that must hold every time that instruction is executed
- Example:
 - **Constant-propagation analysis**: checks if for each point in the program and for each variable used by the program, whether the variable has a unique constant value at that point
 - **Liveliness analysis**: checks for each point in the program, whether the value hold by a particular variable at that point is sure to be overwritten before it is read

Principle Sources of Optimization

Principle Sources of Optimization

- Compiler must preserve the **semantics** of the original program
- A compiler cannot understand enough about a program to replace with an efficient algorithm!
- A compiler knows only how to apply relatively **low-level semantic transformations** such that performing the same operation on the same values yields the same result.
- Examples -
 - Use algebraic identities such as replacing $i+0$ with i
 - Remove redundant operations and codes

Causes of Redundancy

Causes of Redundancy

- There are many redundant operations in a typical program.
- Sometime redundancies are introduced at the source level by the programmer for convenience
 - Cut & paste
- More often the redundancy is a side effect of writing code in high level language such as an array access, $A[i][j]$
 - During compilation, such high-level data-structure accesses expands into a number of low-level arithmetic operations, such as the computation of the location of the (i, j) th element of a matrix A .

Example

Example

Quick Sort(C)

```
void quicksort(int m, int n)
/* recursively sorts a[m] through a[n]
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Example

Quick Sort(C)

```
void quicksort(int m, int n)
/* recursively sorts a[m] through a[n]
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Quick Sort(TAC)

(1)	i = m-1	(16)	t7 = 4*i
(2)	j = n	(17)	t8 = 4*j
(3)	t1 = 4*n	(18)	t9 = a[t8]
(4)	v = a[t1]	(19)	a[t7] = t9
(5)	i = i+1	(20)	t10 = 4*j
(6)	t2 = 4*i	(21)	a[t10] = x
(7)	t3 = a[t2]	(22)	goto (5)
(8)	if t3<v goto (5)	(23)	t11 = 4*i
(9)	j = j-1	(24)	x = a[t11]
(10)	t4 = 4*j	(25)	t12 = 4*i
(11)	t5 = a[t4]	(26)	t13 = 4*n
(12)	if t5>v goto (9)	(27)	t14 = a[t13]
(13)	if i>=j goto (23)	(28)	a[t12] = t14
(14)	t6 = 4*i	(29)	t15 = 4*n
(15)	x = a[t6]	(30)	a[t15] = x

Example

Quick Sort(C)

```
void quicksort(int m, int n)
/* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Quick Sort(TAC)

Basic Block

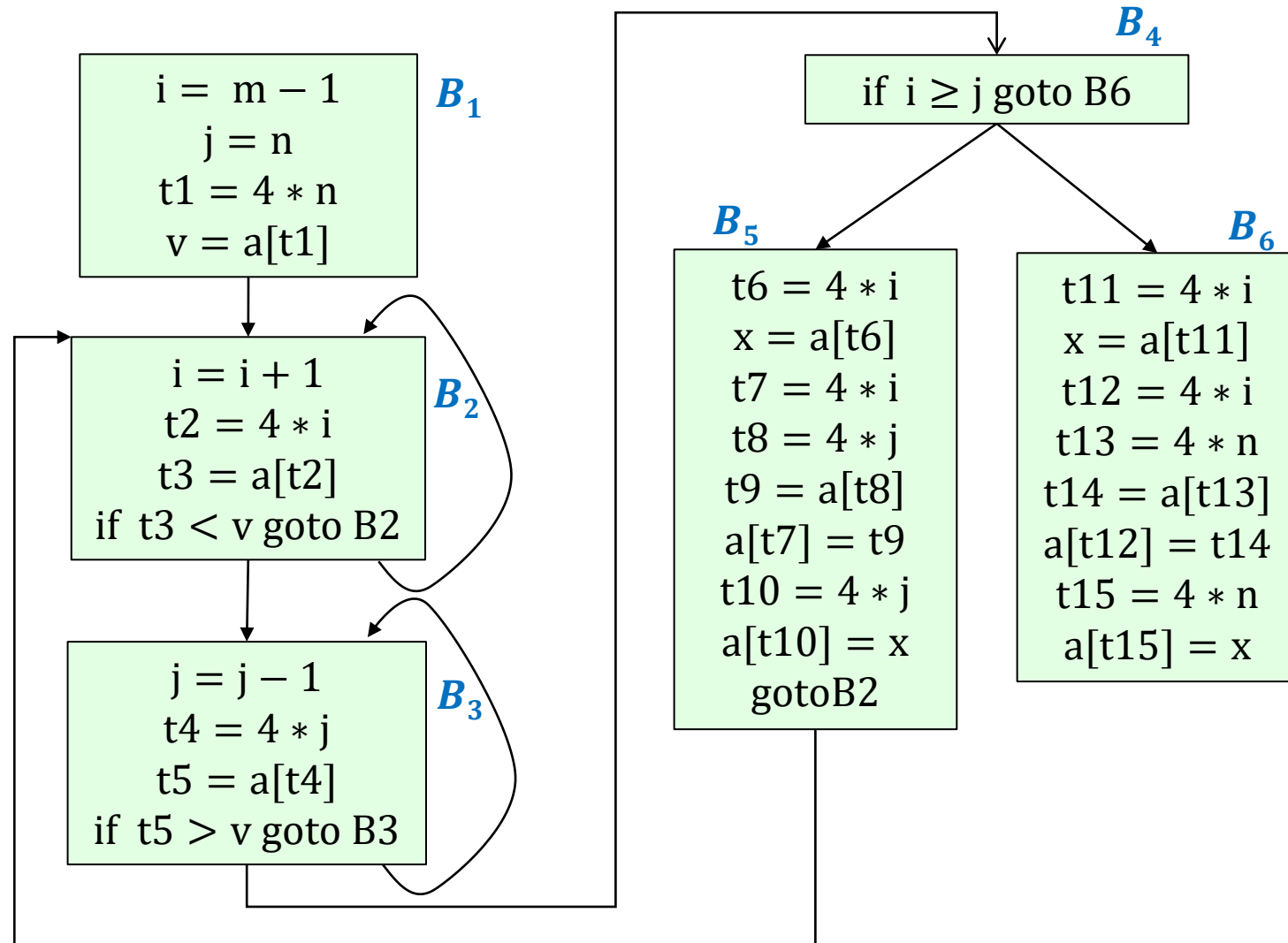
(1)	i = m-1	(16)	t7 = 4*i
(2)	j = n	(17)	t8 = 4*j
(3)	t1 = 4*n	(18)	t9 = a[t8]
(4)	v = a[t1]	(19)	a[t7] = t9
(5)	i = i+1	(20)	t10 = 4*j
(6)	t2 = 4*i	(21)	a[t10] = x
(7)	t3 = a[t2]	(22)	goto (5)
(8)	if t3<v goto (5)	(23)	t11 = 4*i
(9)	j = j-1	(24)	x = a[t11]
(10)	t4 = 4*j	(25)	t12 = 4*i
(11)	t5 = a[t4]	(26)	t13 = 4*n
(12)	if t5>v goto (9)	(27)	t14 = a[t13]
(13)	if i>=j goto (23)	(28)	a[t12] = t14
(14)	t6 = 4*i	(29)	t15 = 4*n
(15)	x = a[t6]	(30)	a[t15] = x

Example

Quick Sort(C)

```
void quicksort(int m, int n)
/* recursively sorts a[m] through a[n] */
{
    int i, j;
    int v, x;
    if (n <= m) return;
    /* fragment begins here */
    i = m-1; j = n; v = a[n];
    while (1) {
        do i = i+1; while (a[i] < v);
        do j = j-1; while (a[j] > v);
        if (i >= j) break;
        x = a[i]; a[i] = a[j]; a[j] = x;
    }
    x = a[i]; a[i] = a[n]; a[n] = x;
    /* fragment ends here */
    quicksort(m,j); quicksort(i+1,n);
}
```

Quick Sort(Flow Graph)



Semantic-Preserving Transformations

Semantic-Preserving Transformations

- Improve code preserving its semantics
- Common Examples:
 - Common subexpression elimination
 - Copy propagation
 - Constant folding
 - Dead-code elimination
 - Code motion
 - Induction variables and reduction in strength

Common-Subexpression Elimination

Common-Subexpression Elimination

- *Local Common Subexpressions Elimination*

Common-Subexpression Elimination

- *Local Common Subexpressions Elimination*

<pre>t6 = 4 * i x = a[t6] t7 = 4 * i t8 = 4 * j t9 = a[t8] a[t7] = t9 t10 = 4 * j a[t10] = x goto B₂</pre>	B_5
---	--------------------------------

Before

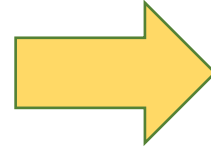
Common-Subexpression Elimination

- *Local Common Subexpressions Elimination*

```
t6 = 4 * i
x = a[t6]
t7 = 4 * i
t8 = 4 * j
t9 = a[t8]
a[t7] = t9
t10 = 4 * j
a[t10] = x
goto B2
```

B₅

Before

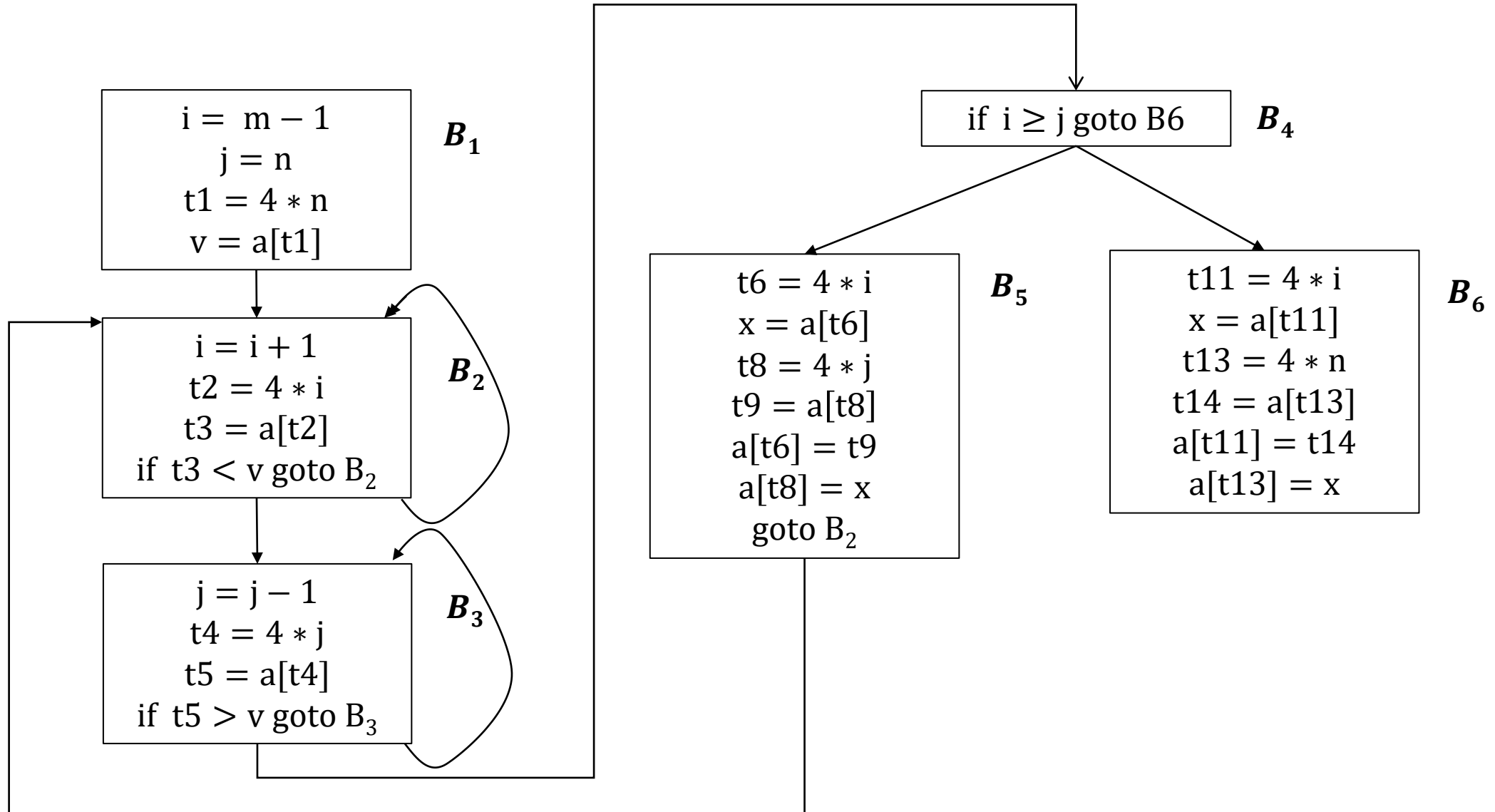


```
t6 = 4 * i
x = a[t6]
t8 = 4 * j
t9 = a[t8]
a[t6] = t9
a[t8] = x
goto B2
```

B₅

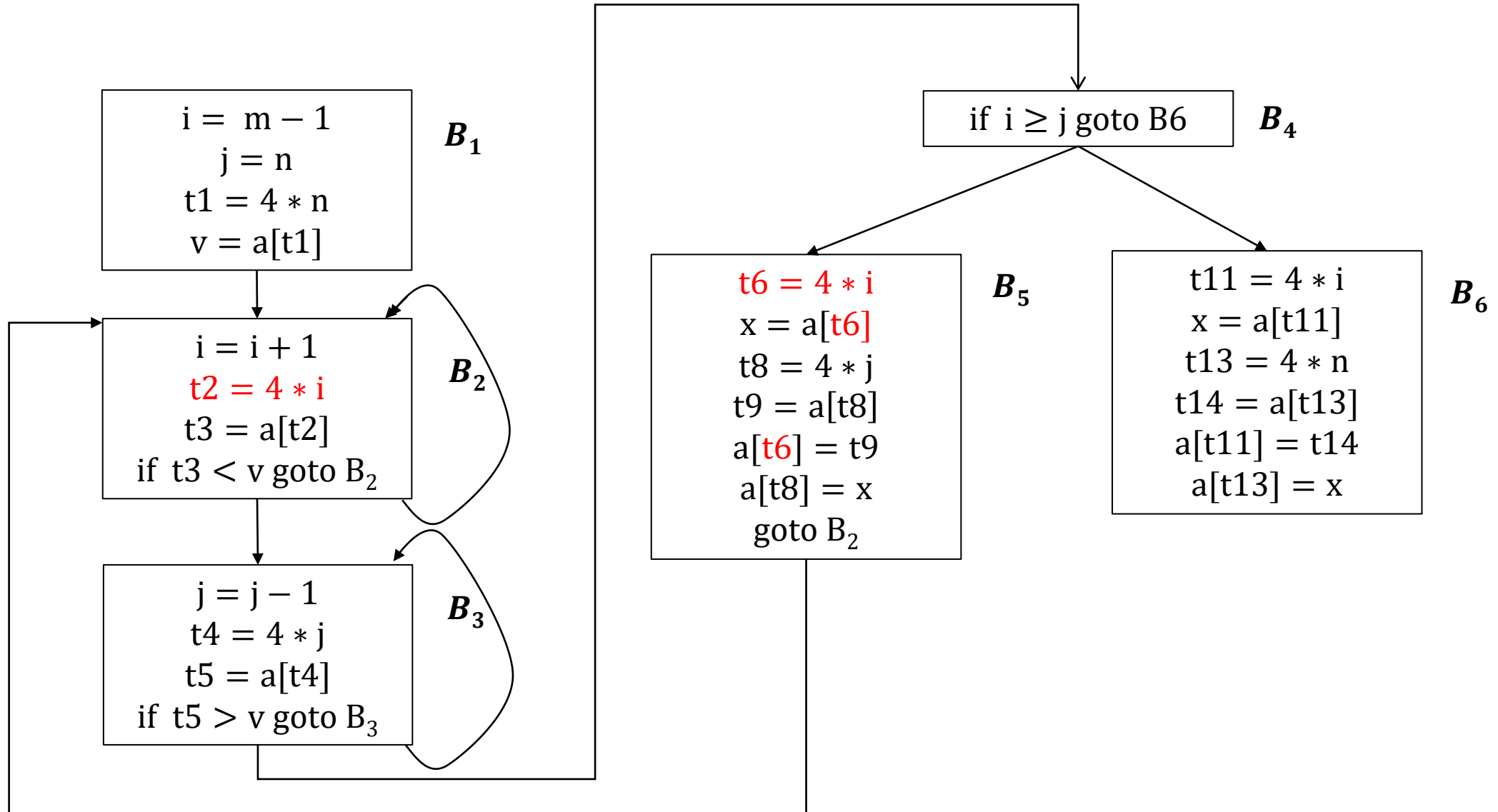
After

Global Common Subexpressions



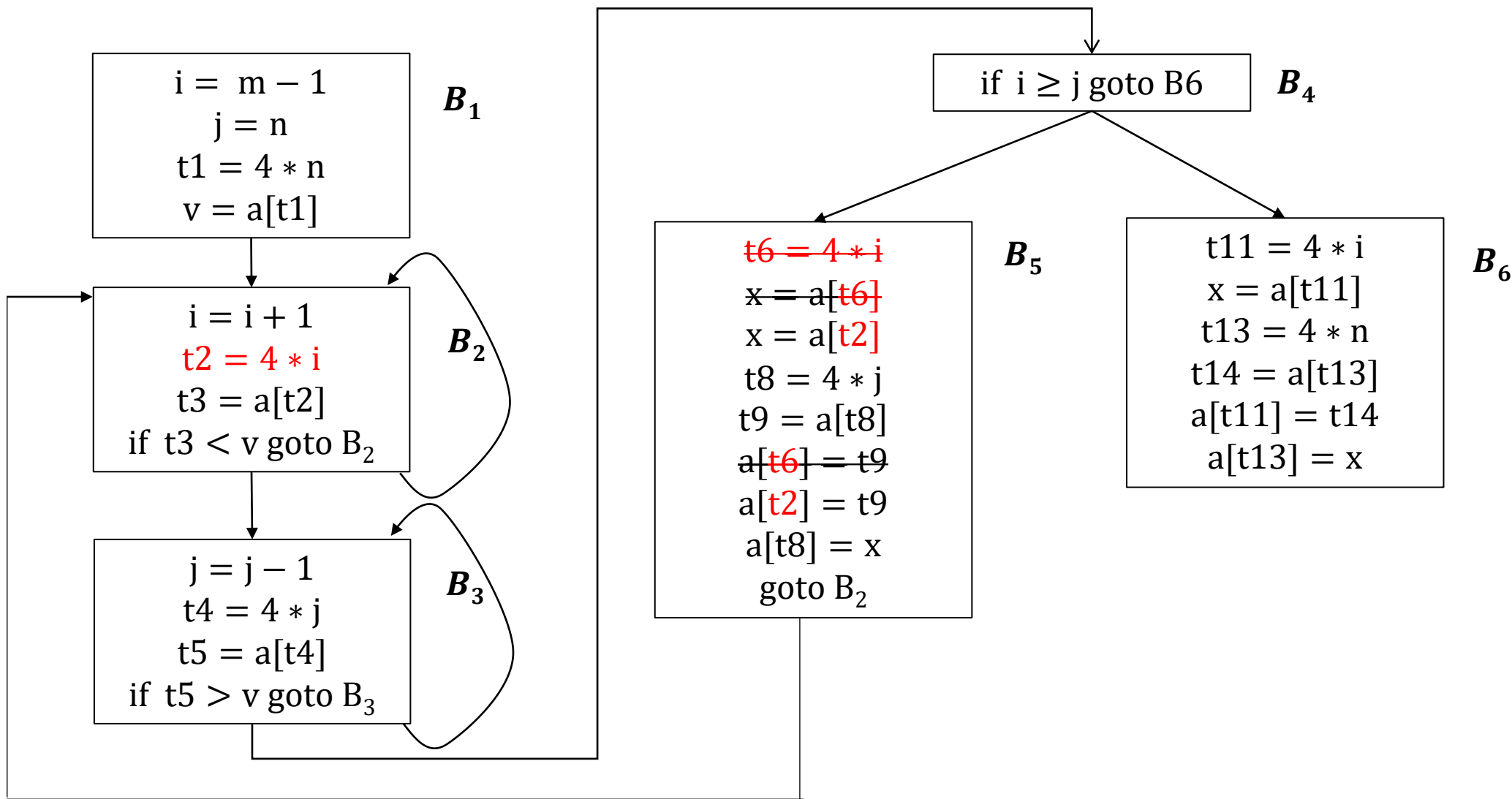
Global Common Subexpressions

i and t2 do not change when block B5 starts to execute

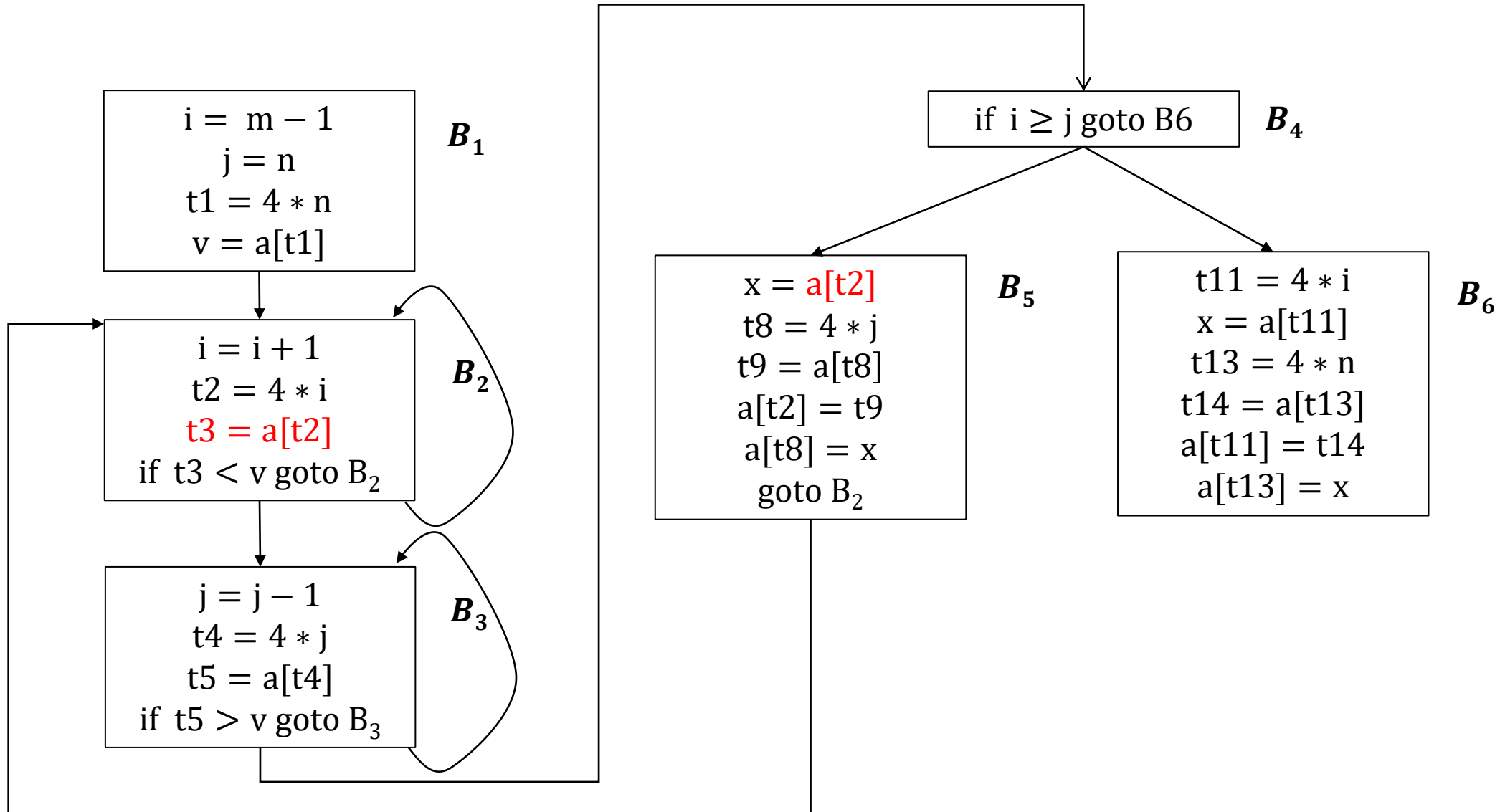


Global Common Subexpressions

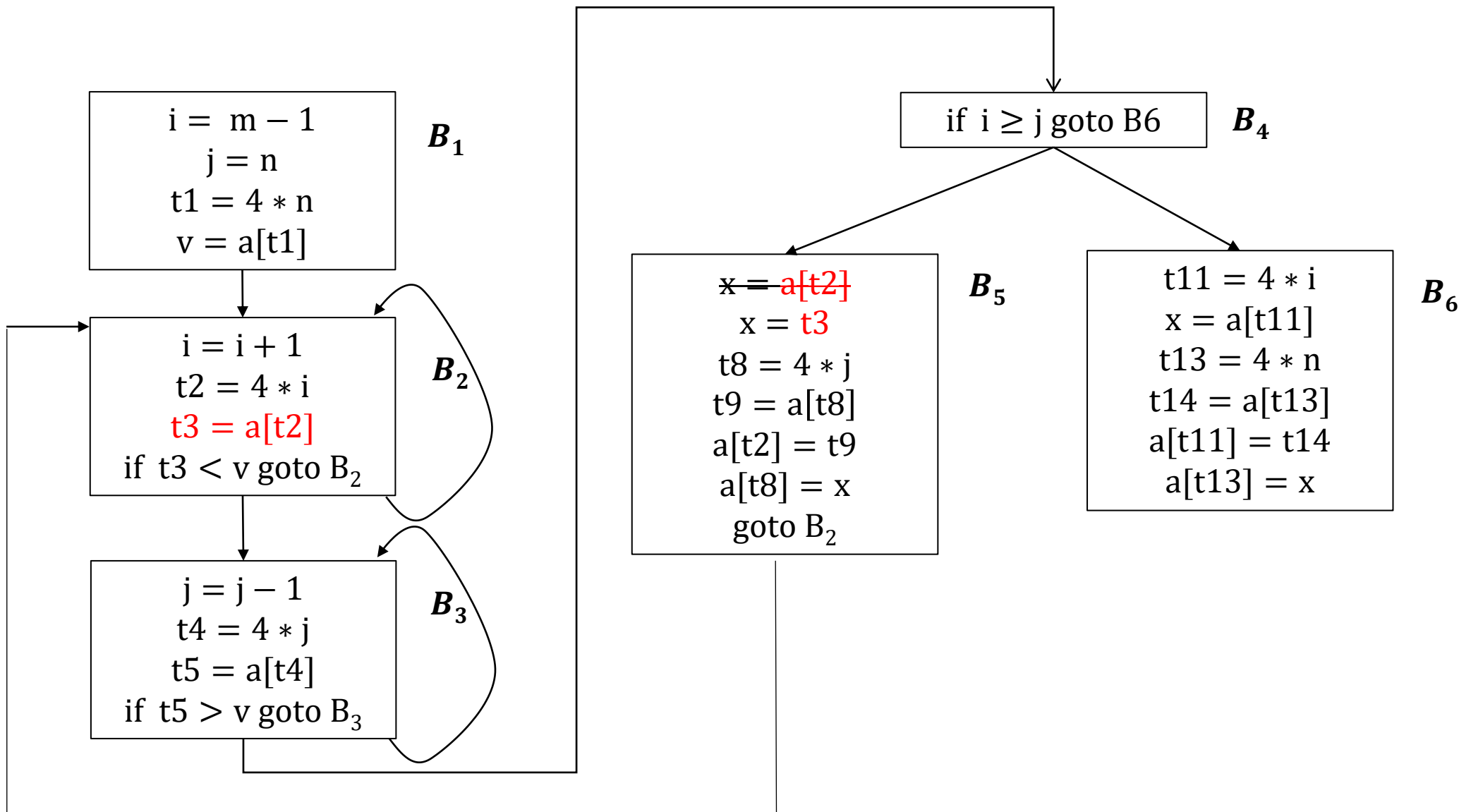
No need to calculate t6 and t6 is replaced with t2



Global Common Subexpressions

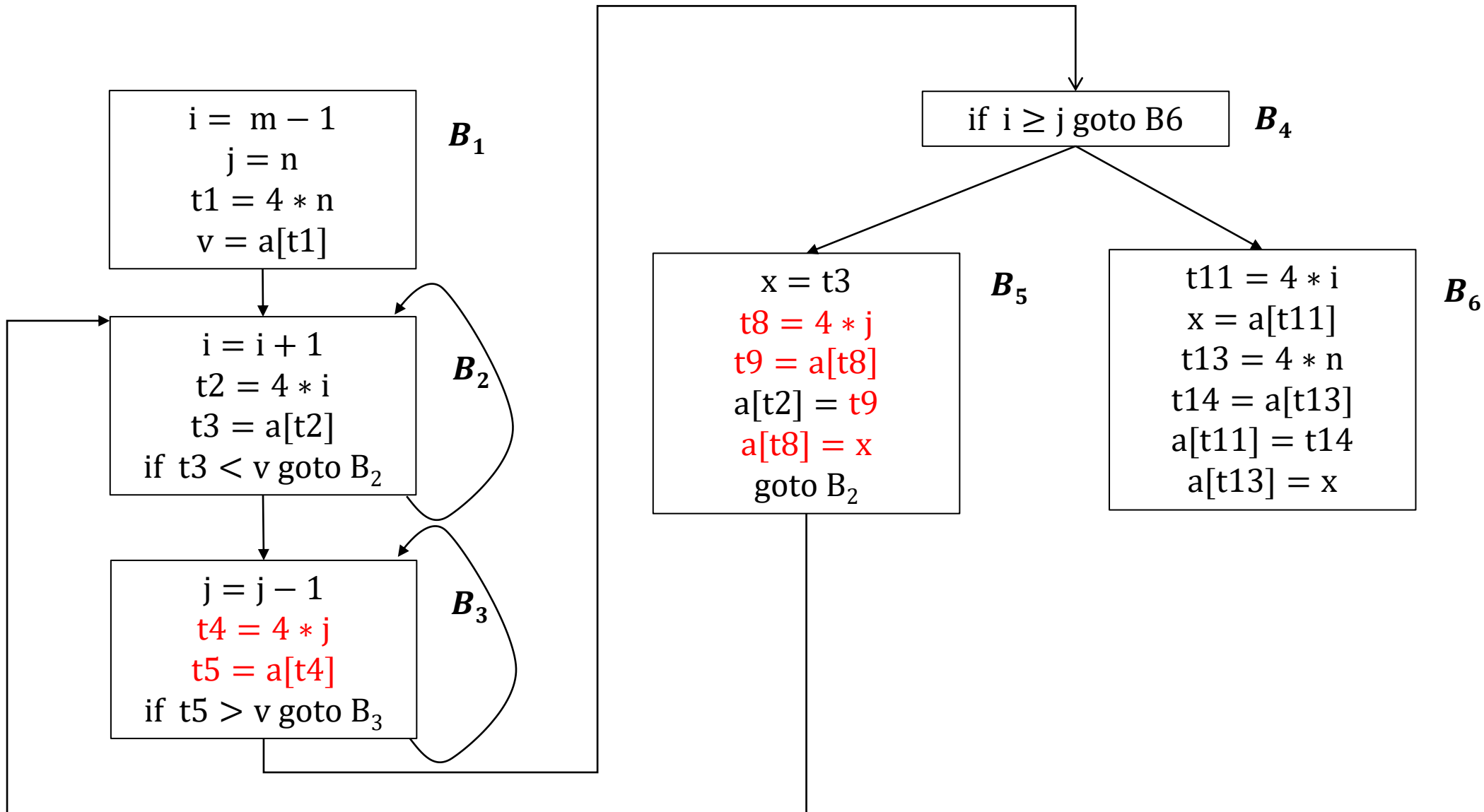


Global Common Subexpressions



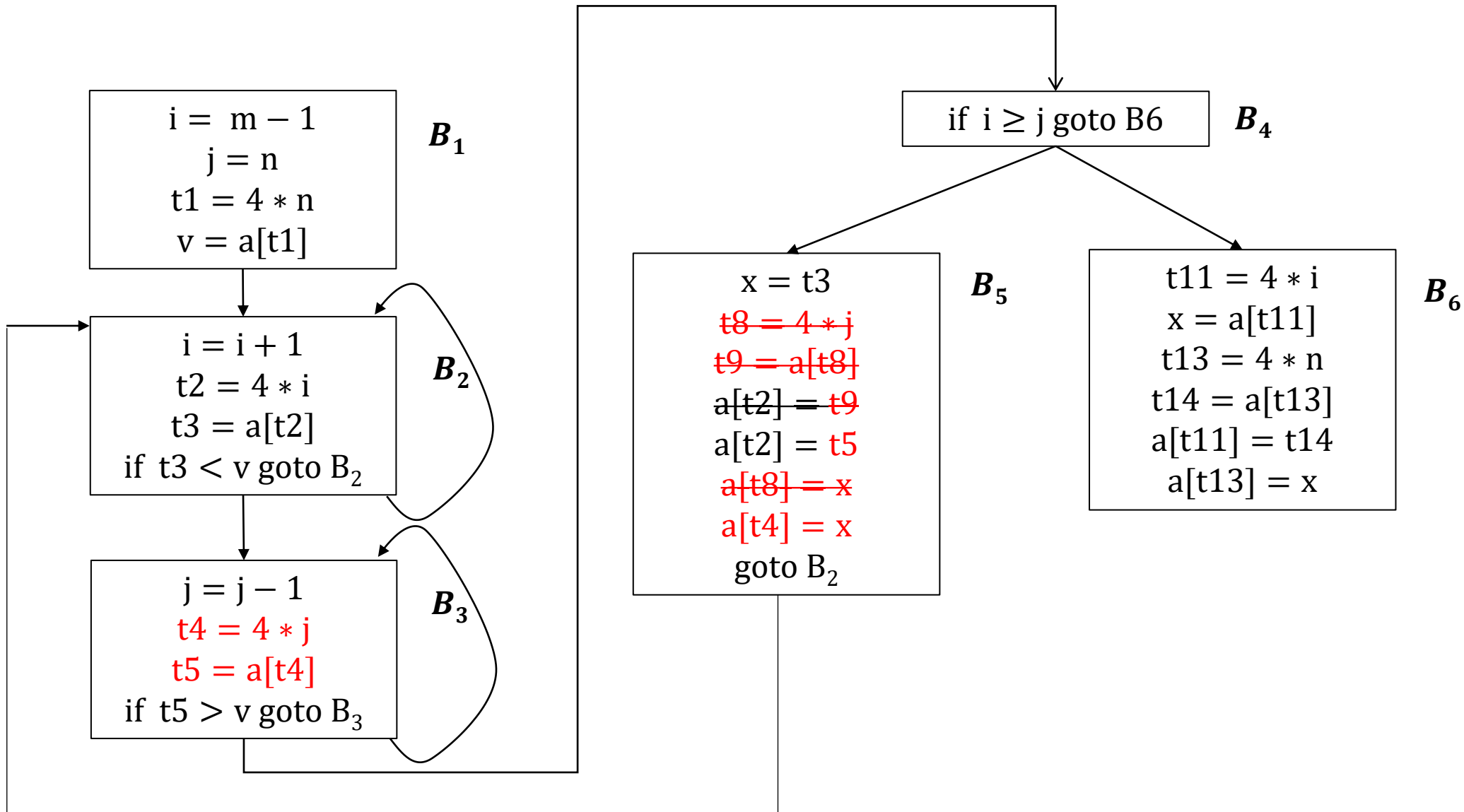
Global Common Subexpressions

j and t4 do not change when block B5 starts to execute

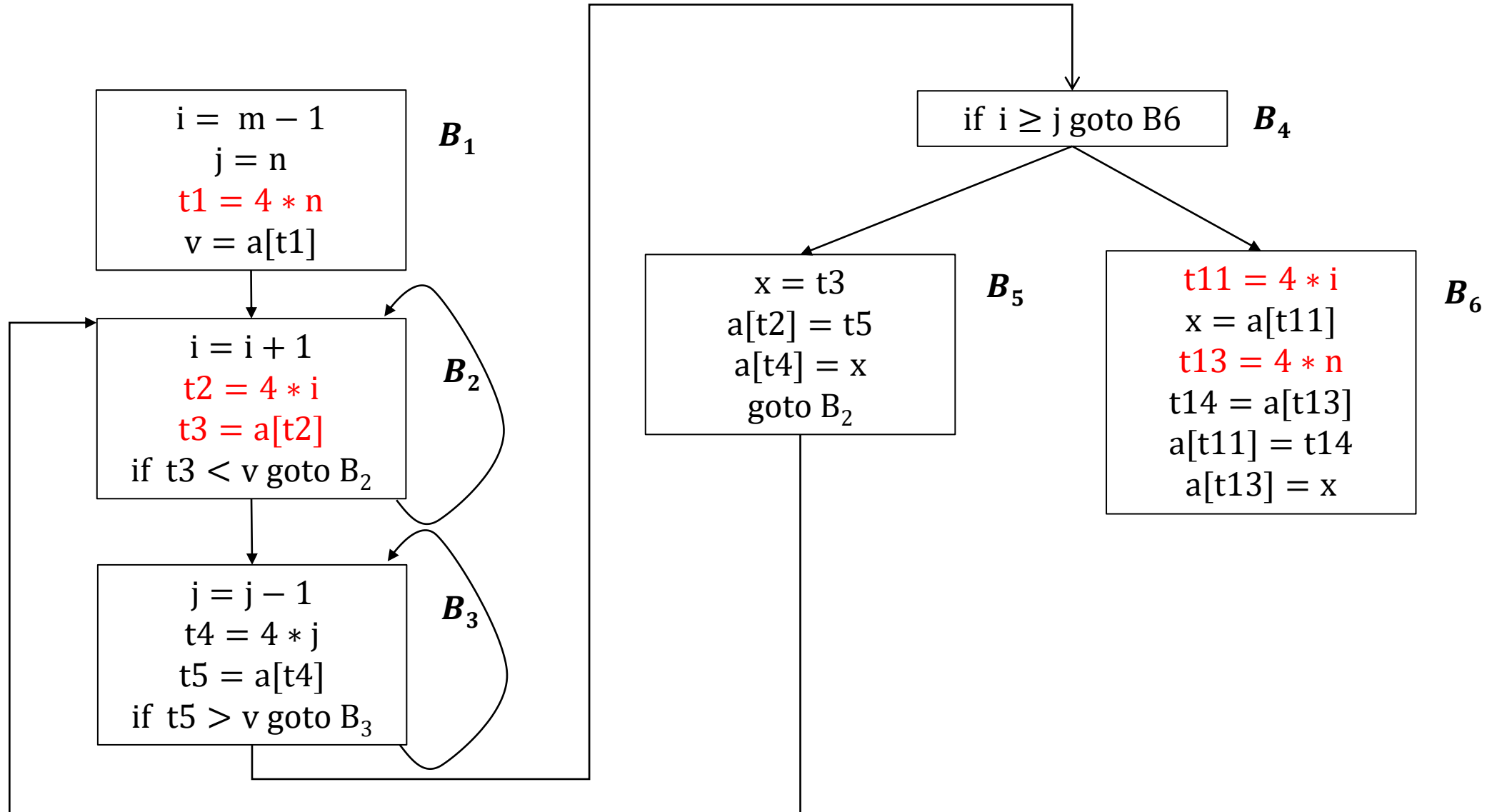


Global Common Subexpressions

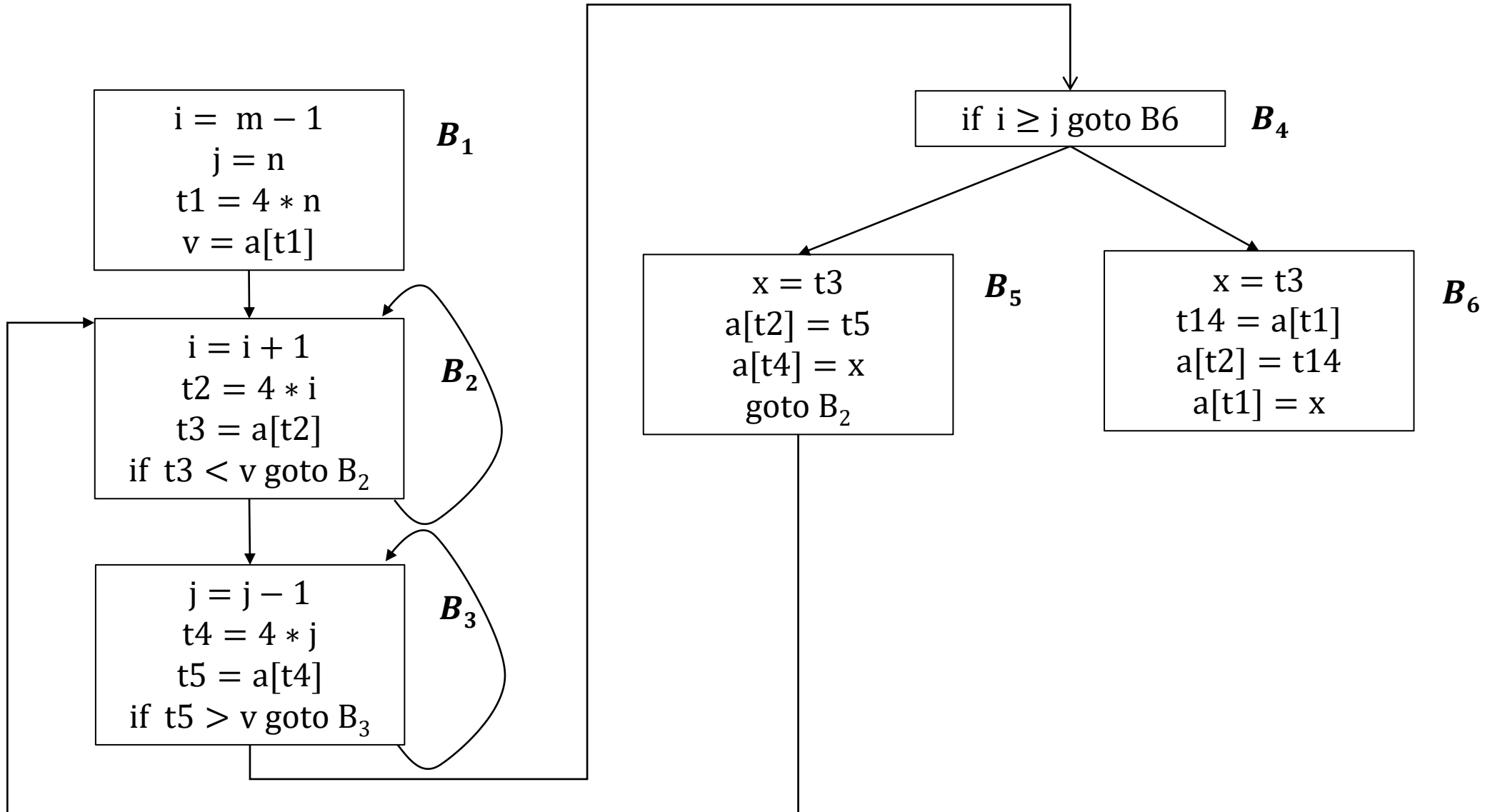
No need to calculate t8 and t8 is replaced with t4



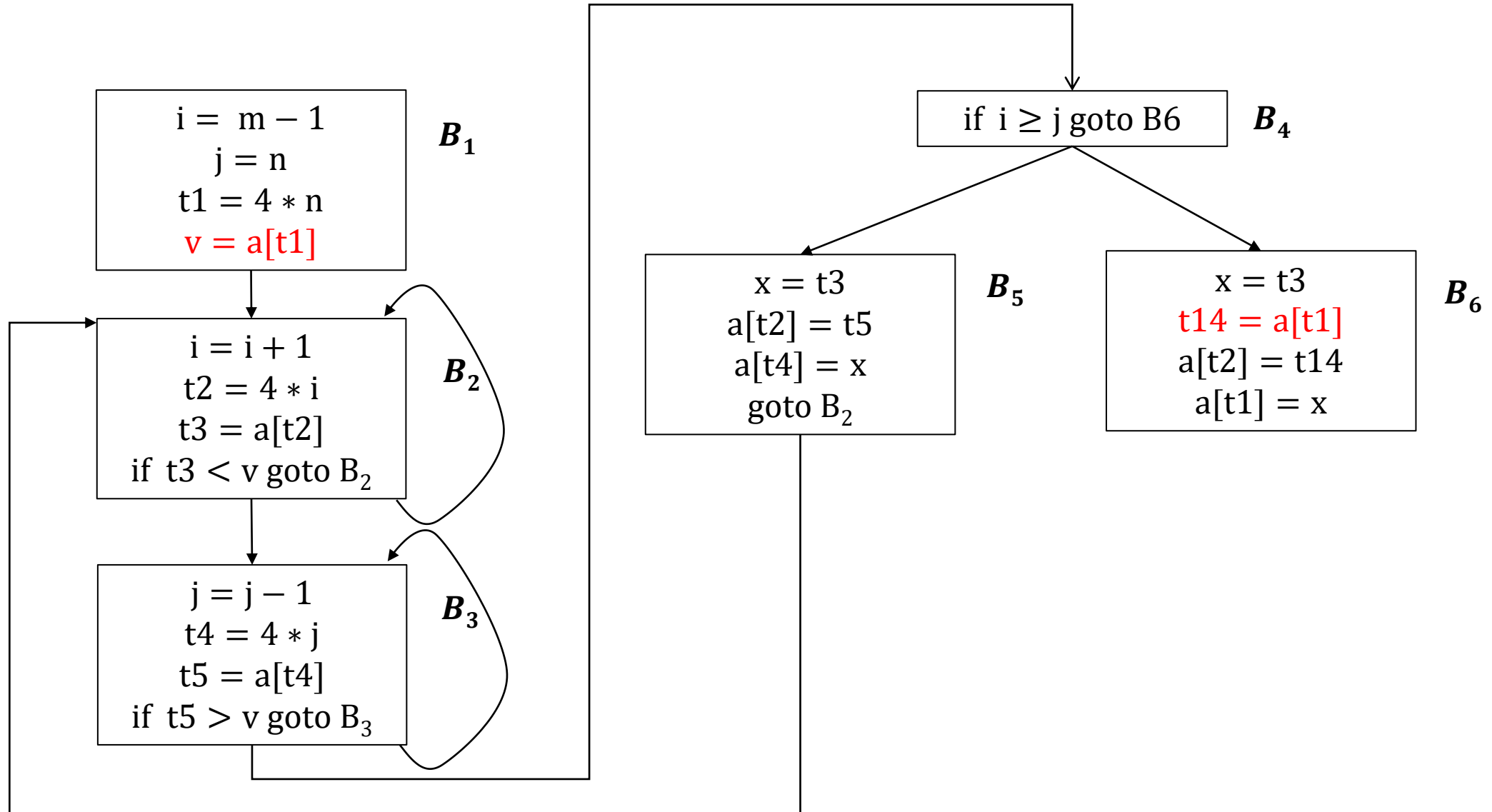
Global Common Subexpressions



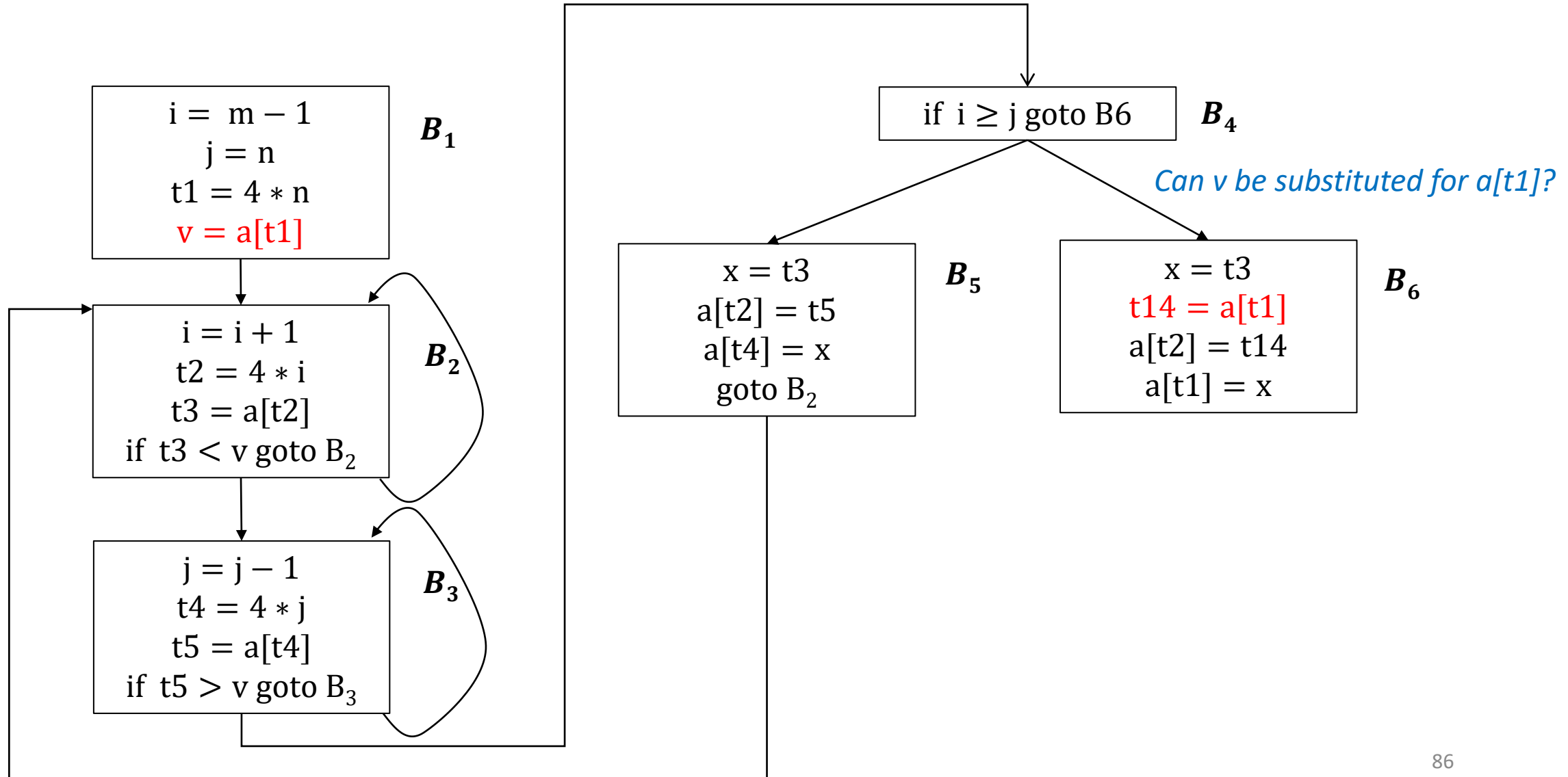
Global Common Subexpressions



Global Common Subexpressions



Global Common Subexpressions



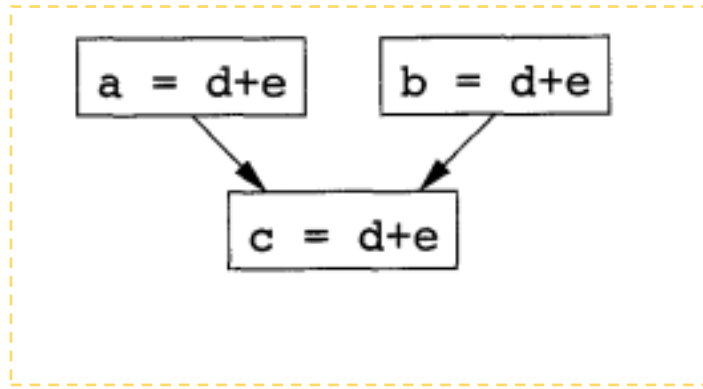
Copy Propagation

Copy Propagation

- Copies are introduced during common subexpression elimination

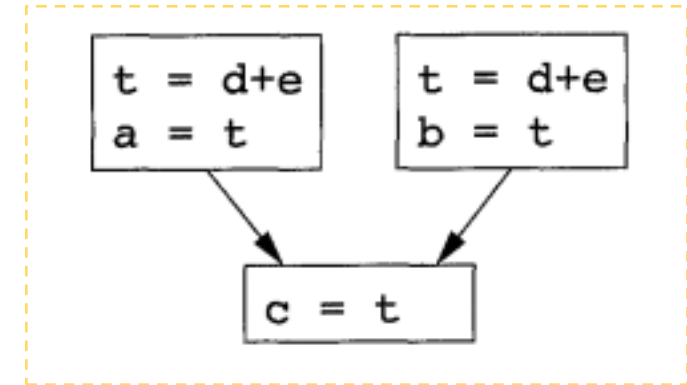
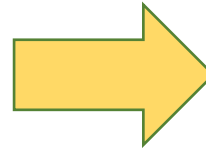
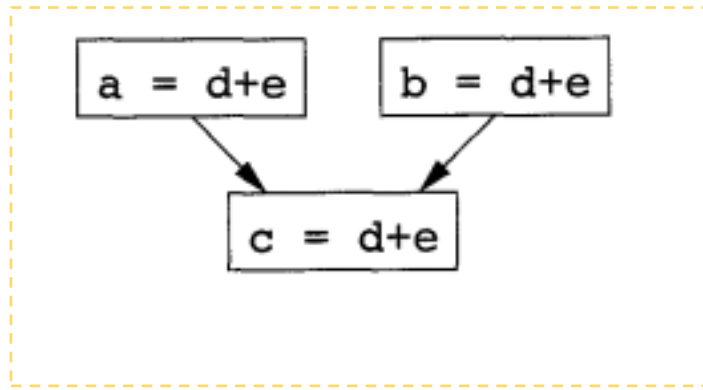
Copy Propagation

- Copies are introduced during common subexpression elimination



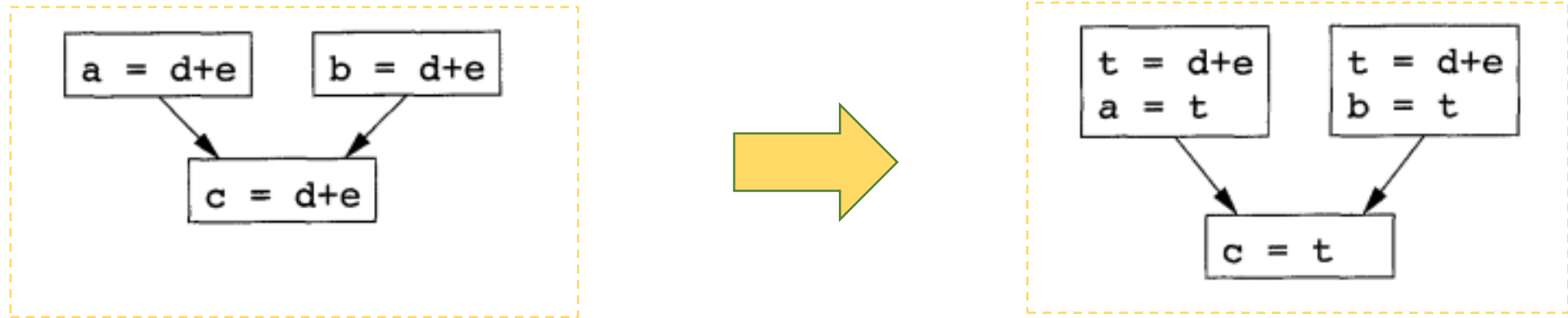
Copy Propagation

- Copies are introduced during common subexpression elimination



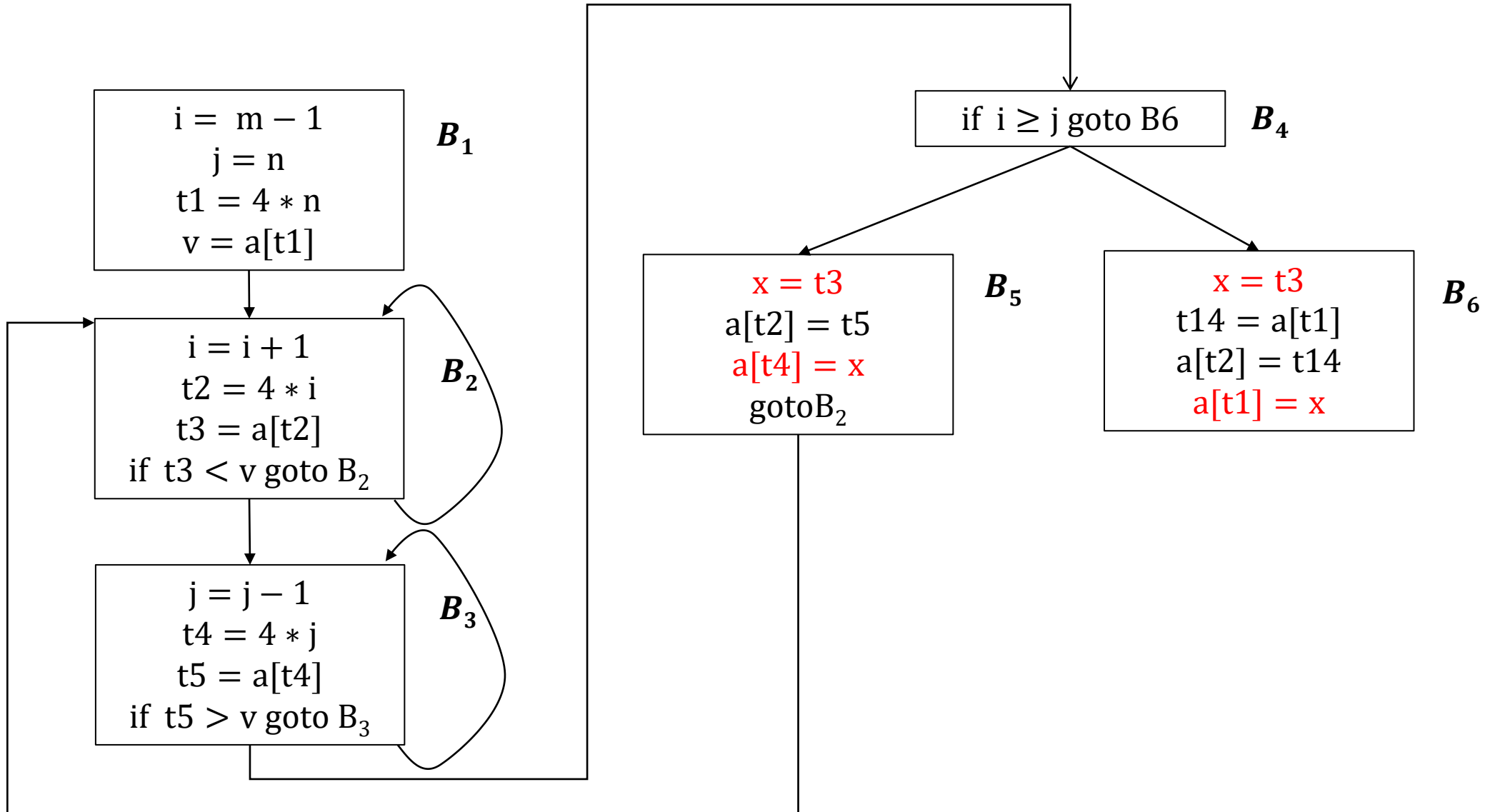
Copy Propagation

- Copies are introduced during common subexpression elimination

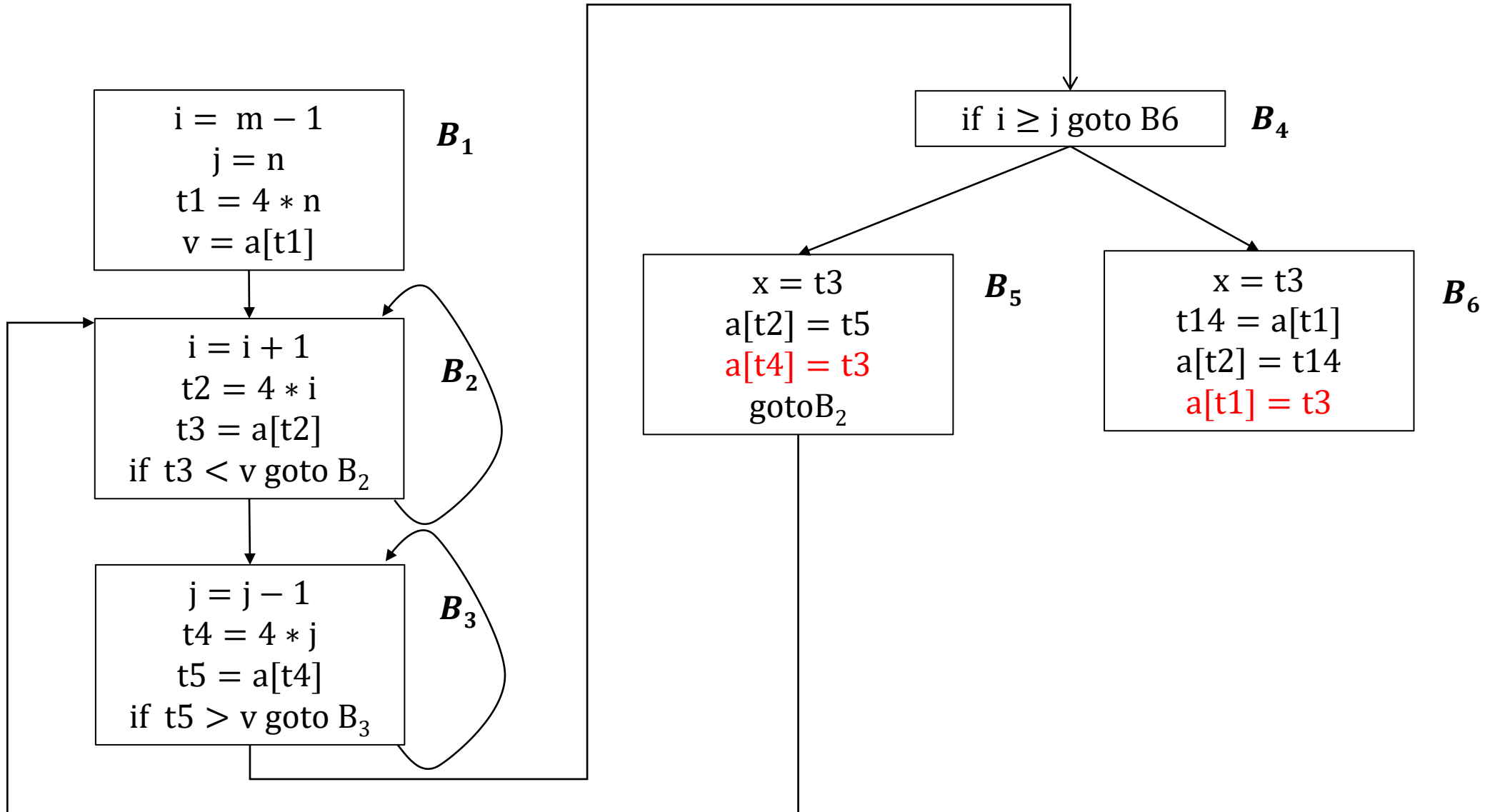


- The idea behind **copy-propagation** transformation is to use *t* for *c* whenever possible after the copy statement *c=t*

Copy Propagation



Copy Propagation



Dead Code Elimination

Dead Code Elimination

- *Dead Code*: Statements that computes values that never get used
- May result from various transformations, such as-
 - *Constant folding*: deducing at compile time that the value of an expression is a constant and using the constant instead
 - *Copy propagation*
- Example:

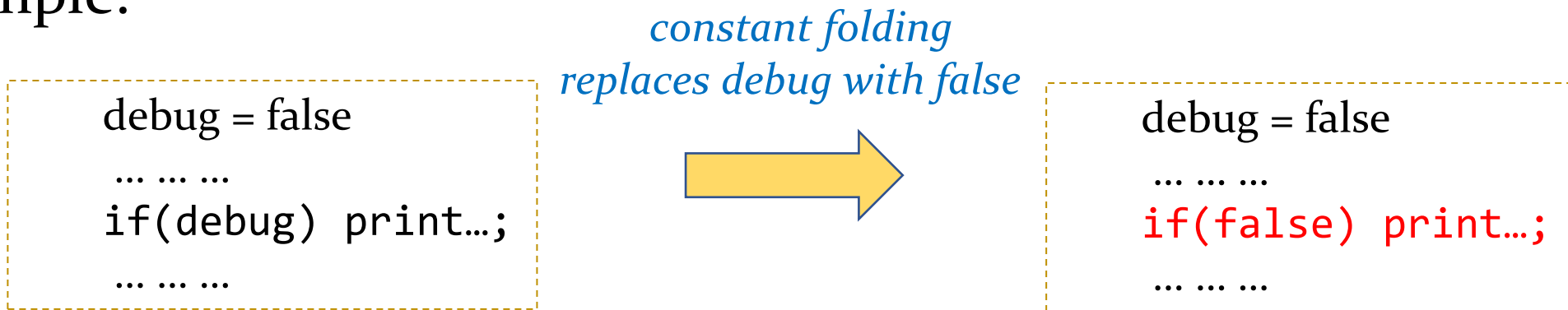
Dead Code Elimination

- *Dead Code*: Statements that computes values that never get used
- May result from various transformations, such as-
 - *Constant folding*: deducing at compile time that the value of an expression is a constant and using the constant instead
 - *Copy propagation*
- Example:

```
debug = false  
... ..  
if(debug) print...;  
... ..
```

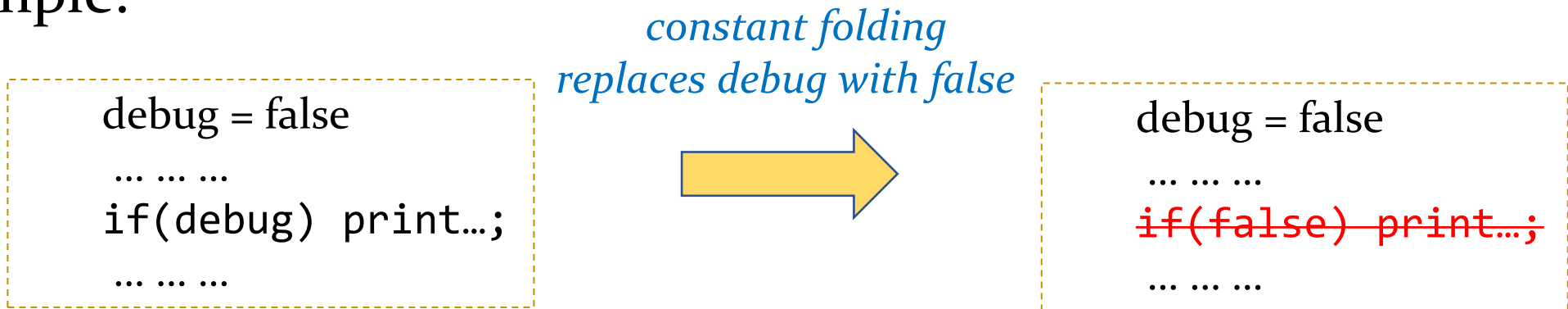
Dead Code Elimination

- *Dead Code*: Statements that computes values that never get used
- May result from various transformations, such as-
 - *Constant folding*: deducing at compile time that the value of an expression is a constant and using the constant instead
 - *Copy propagation*
- Example:

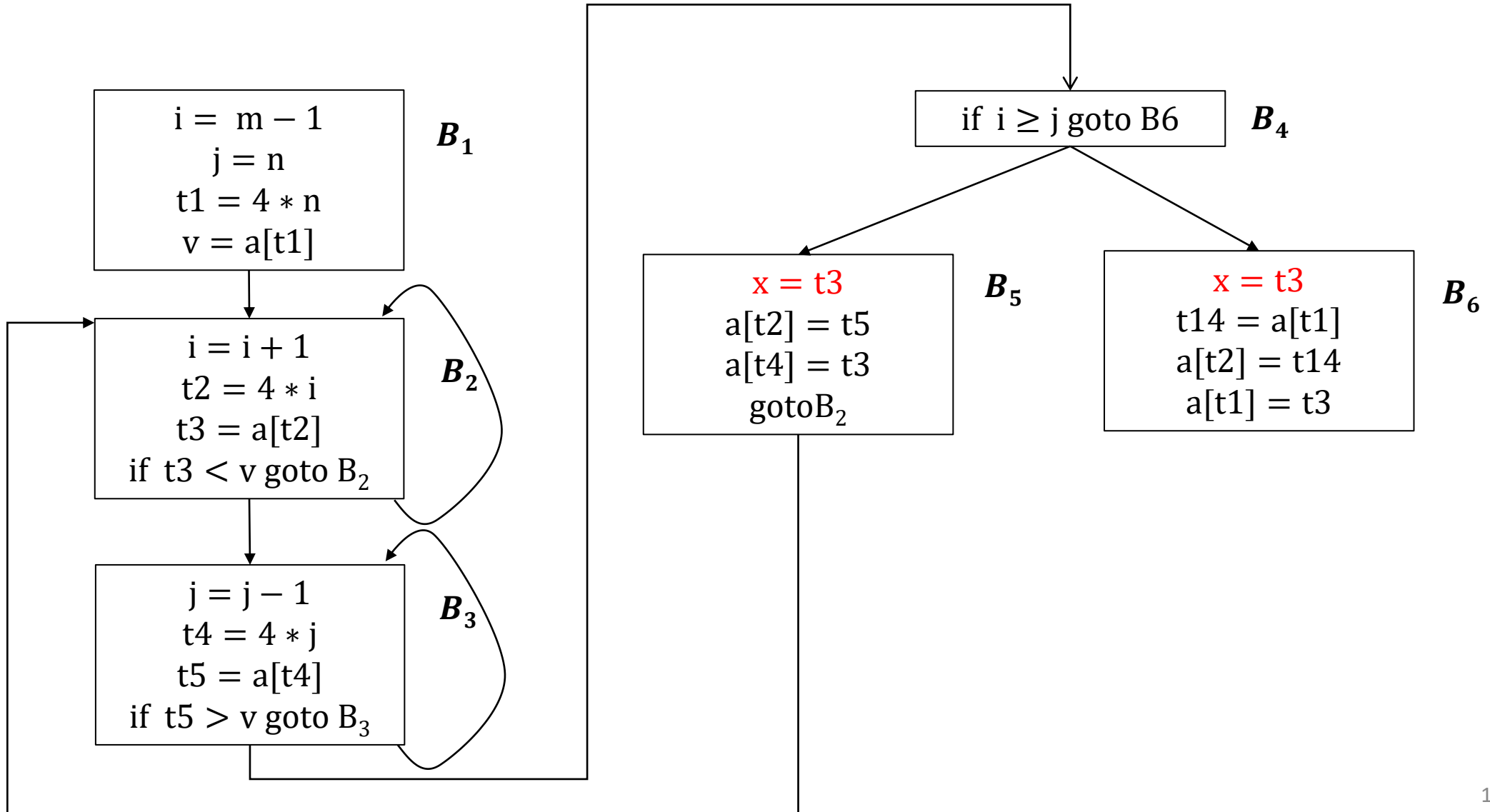


Dead Code Elimination

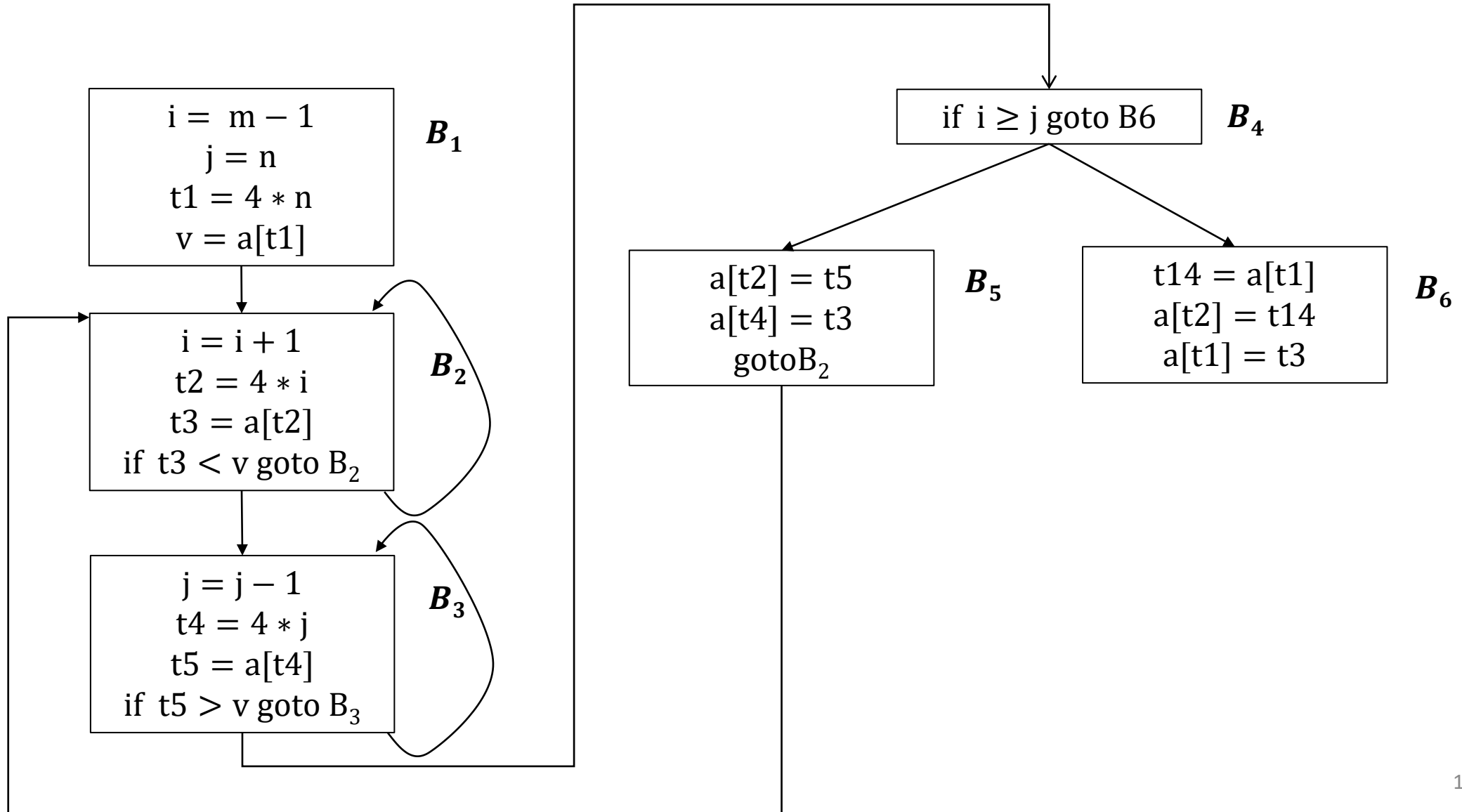
- *Dead Code*: Statements that computes values that never get used
- May result from various transformations, such as-
 - *Constant folding*: deducing at compile time that the value of an expression is a constant and using the constant instead
 - *Copy propagation*
- Example:



Dead Code Elimination



Dead Code Elimination



Code Motion

Code Motion

- Reducing the amount of code in a **loop** may improve running time of a program significantly
 - Specially inner loops
- *Code Motion*
 - Moves **loop-invariant computations** out the loop and evaluates before the loop.
- Example:

Code Motion

- Reducing the amount of code in a **loop** may improve running time of a program significantly
 - Specially inner loops
- *Code Motion*
 - Moves **loop-invariant computations** out the loop and evaluates before the loop.
- Example:

```
while (i <= limit-2) /* statement does not change limit */
```

Code Motion

- Reducing the amount of code in a **loop** may improve running time of a program significantly
 - Specially inner loops
- *Code Motion*
 - Moves **loop-invariant computations** out the loop and evaluates before the loop.
- Example:

```
while (i <= limit-2) /* statement does not change limit */
```



```
t = limit-2  
while (i <= t) /* statement does not change limit or t */
```

Induction Variables & Strength Reduction

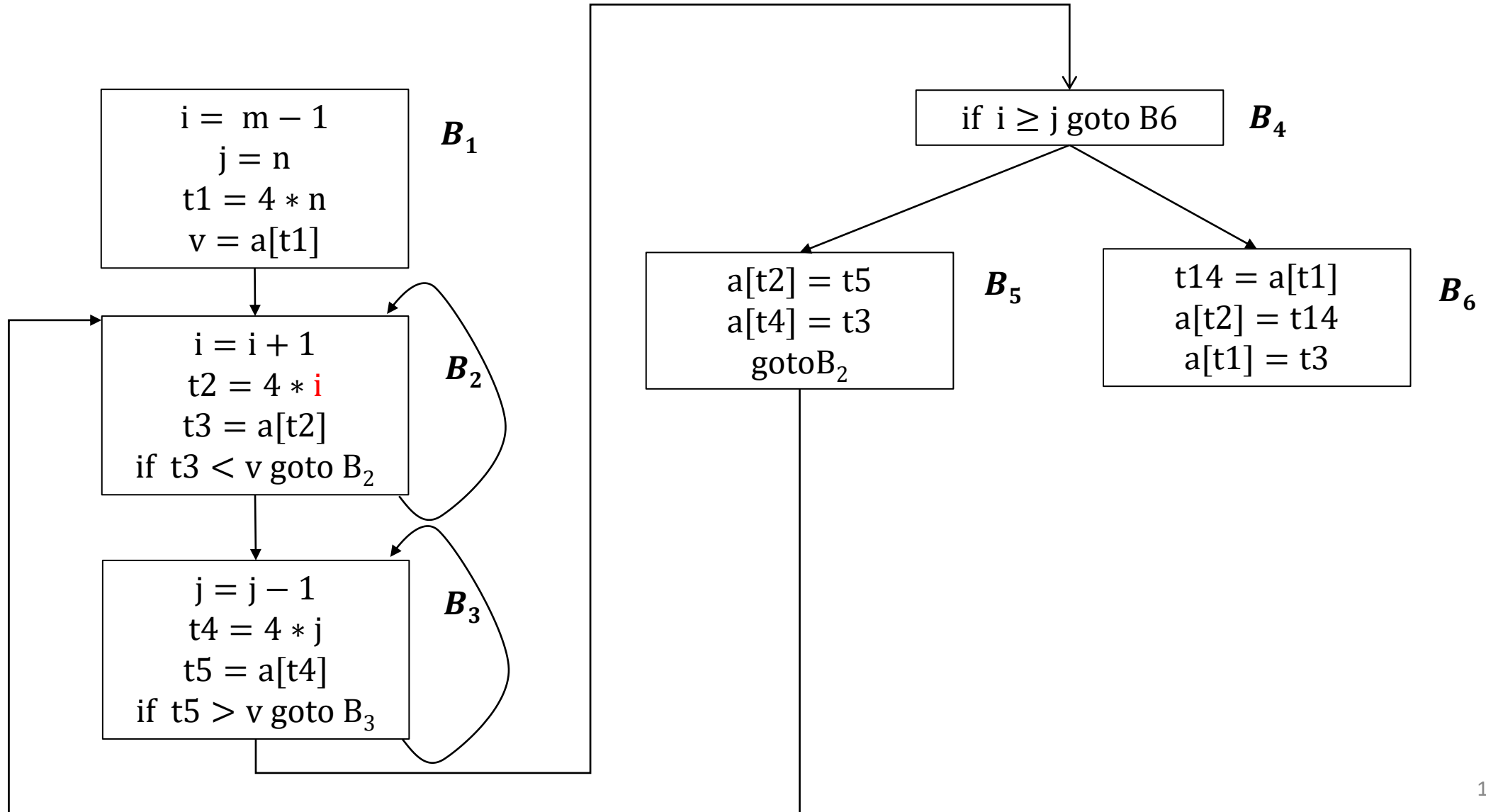
- Induction Variables

- A variable which is increased by a constant(+ve or -ve) c , every time the variable is assigned

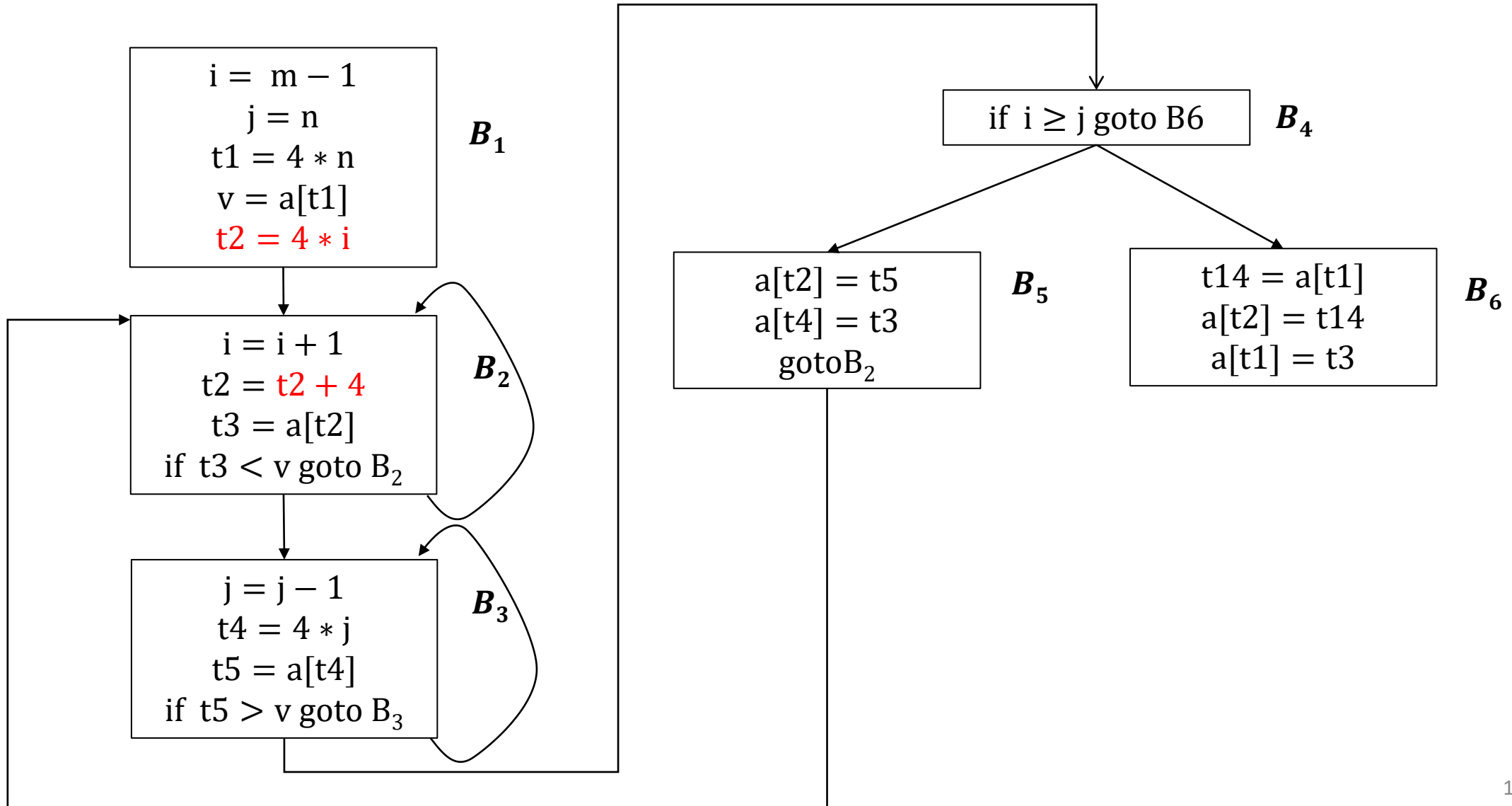
- Strength Reduction

- Replacing an expensive operation by a cheaper one
- Multiplication by addition

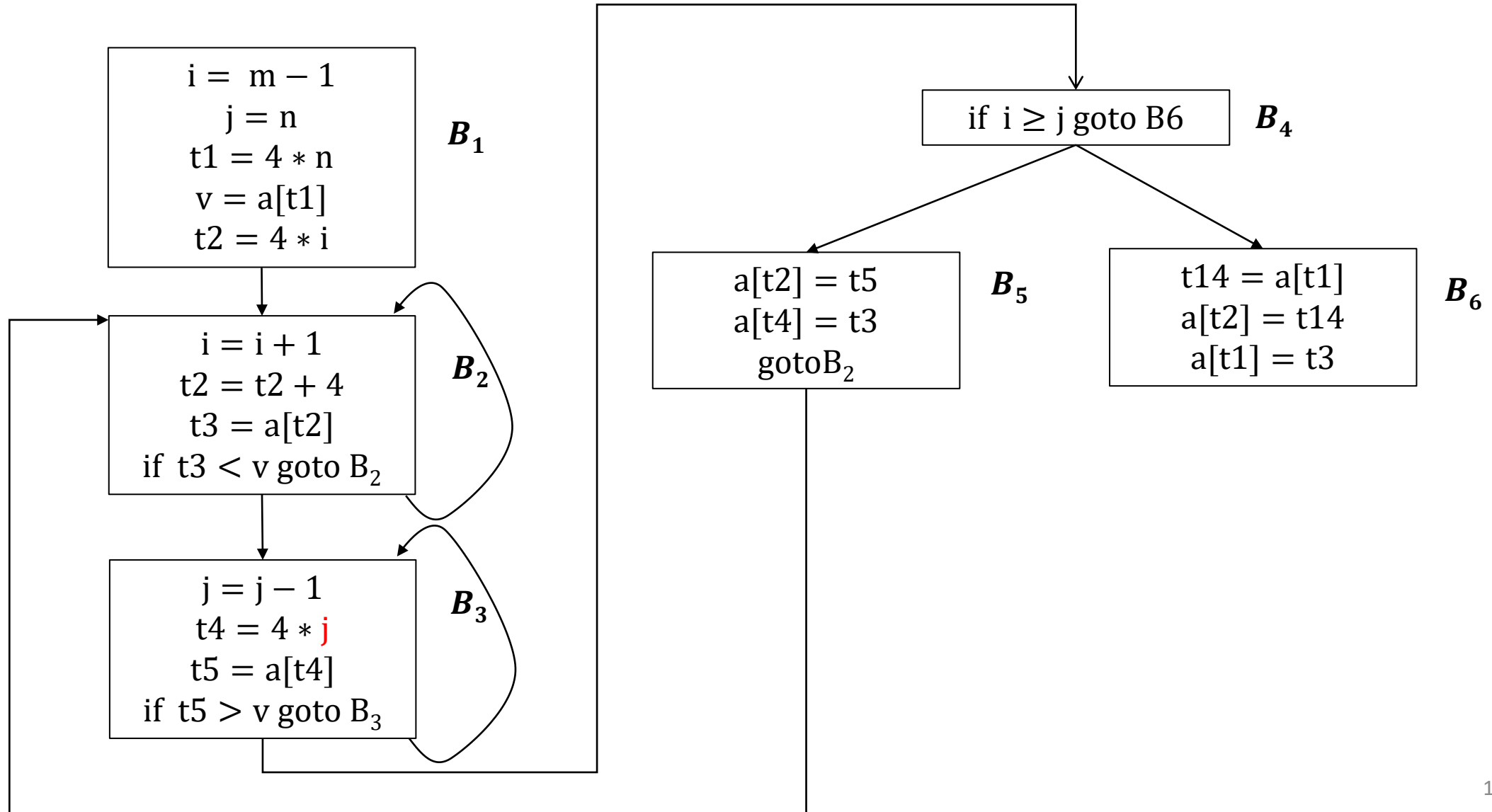
Induction Variables & Strength Reduction



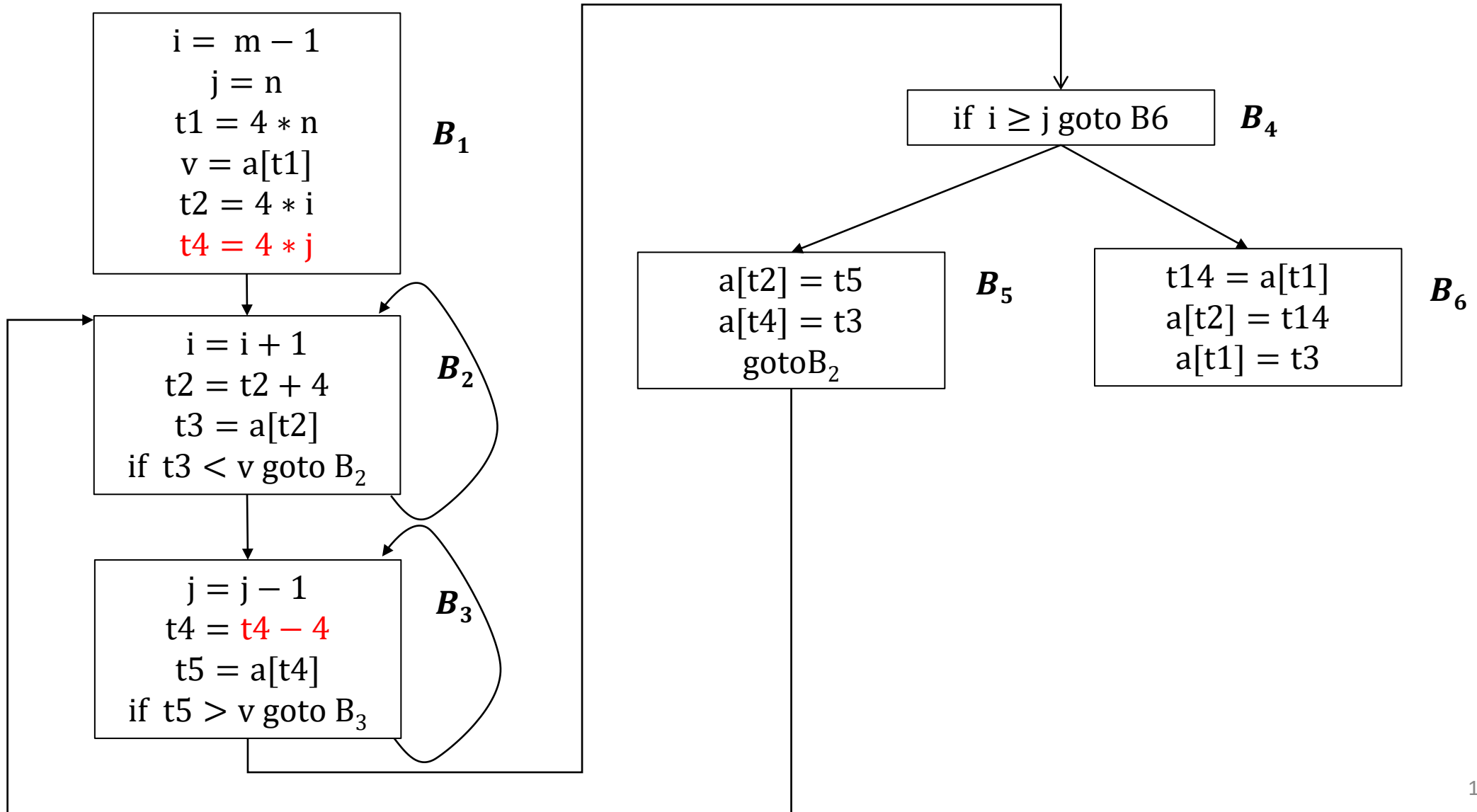
Induction Variables & Strength Reduction



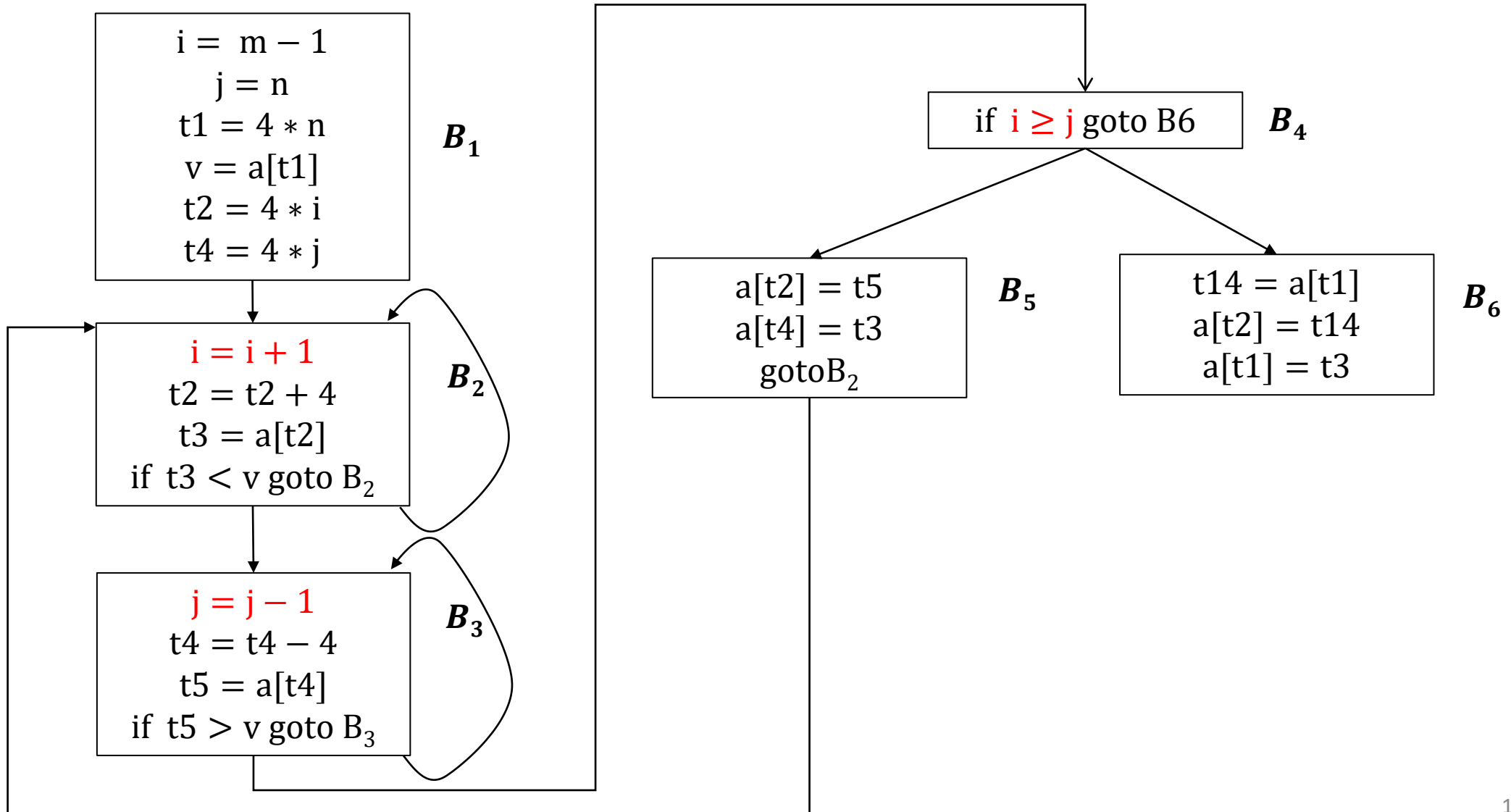
Induction Variables & Strength Reduction



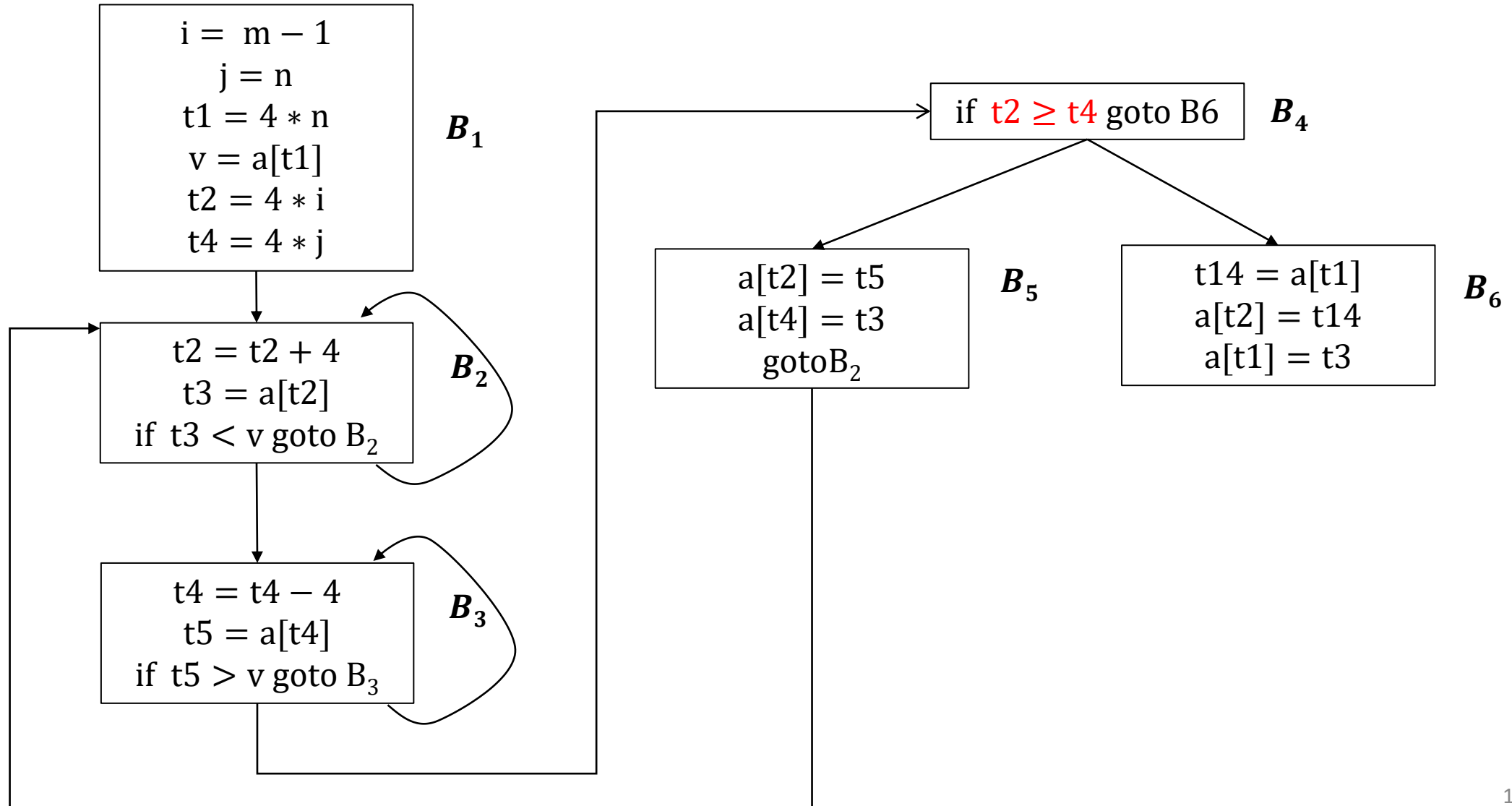
Induction Variables & Strength Reduction



Induction Variables & Strength Reduction

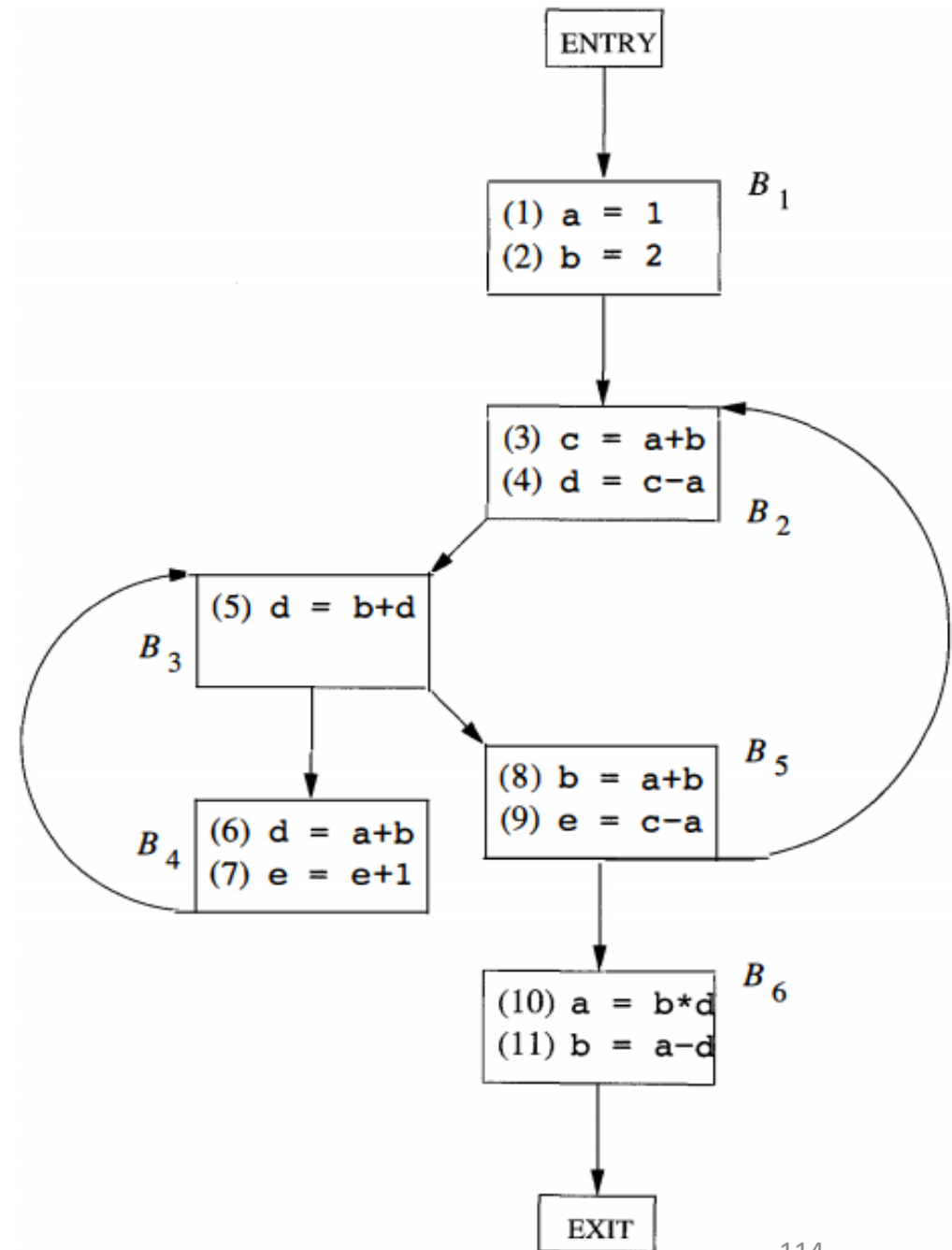


Induction Variables & Strength Reduction



Exercise

- Identify the loops of the flow graph.
- Statements (1) and (2) in B_1 are both copy statements, in which a and b are given constant values. For which uses of a and b can we perform copy propagation and replace these uses of variables by uses of a constant? Do so, wherever possible.
- Identify any global common subexpressions for each loop.
- Identify any induction variables for each loop. Be sure to take into account any constants introduced in (b).
- Identify any loop-invariant computations for each loop.



Reference

- Compilers: Principles, Techniques, & Tools, by
Alfred B. Aho, Monica S. Lam, Ravi Sethi, Jeffrey D. Ullman
 - Chapter 9
 - 9.1

Thank You!